

DESARROLLO PARA DISPOSITIVOS INTELIGENTES

PRACTICA 2.6

Monitor de Actividad Física

Wearable (Wear OS) → Teléfono (Flutter)

Unidad	Duración	Dificultad	Puntos
Unidad 2 - Flutter + Wear OS	4-5 horas	Media-Alta	100 pts.

Cuatrimestre Mayo – Agosto de 2026 - IDGS

1. OBJETIVO

En esta práctica el wearable (Wear OS emulado) será el origen de los datos. Simulara sensores de actividad física generando pasos, calorías y ritmo cardiaco cada segundo, y los enviara vía Bluetooth Low Energy (BLE) a la app Flutter del teléfono, donde se visualizaran en tiempo real mediante un widget de monitoreo.

1.1 Flujo de datos

Origen	Dato generado	Destino	Como llega
Wearable (Wear OS)	Pasos acumulados	Teléfono (Flutter)	BLE NOTIFY
Wearable (Wear OS)	Ritmo cardiaco (bpm)	Teléfono (Flutter)	BLE NOTIFY
Wearable (Wear OS)	Calorías quemadas	Teléfono (Flutter)	BLE NOTIFY
Wearable (Wear OS)	Estado actividad	Teléfono (Flutter)	BLE NOTIFY

Diferencia clave con P2.4

En P2.4 el teléfono ENVIABA datos al wearable (temperatura desde la API).

En P2.6 el wearable GENERA y ENVIA datos al teléfono (sensores de actividad).

Esto requiere usar NOTIFY en lugar de WRITE: el wearable notifica al teléfono cada vez que hay un nuevo valor, sin que el teléfono tenga que pedirlo.

1.2 Objetivos específicos

1. Crear app Wear OS que simule sensores y exponga datos vía BLE NOTIFY.
2. Crear widget de monitoreo en Flutter con tarjetas animadas para cada métrica.
3. Implementar ActivityProvider que recibe y acumula los datos del wearable.
4. Manejar correctamente el ciclo de vida de la suscripción BLE (suscribir, recibir, cancelar).
5. Mostrar alerta cuando el ritmo cardiaco supere 120 bpm.

2. PREPARACION DEL ENTORNO

2.1 Emuladores necesarios

Necesitaras dos emuladores corriendo al mismo tiempo. Si los creaste en P2.4 ya están listos; de lo contrario sigue estos pasos.

Emulador de teléfono

1. En Android Studio abre Device Manager (icono de teléfono en la barra lateral). 2. Crea un Pixel 6 con API 34 (Android 14) si no tienes uno. Asígnale al menos 2 GB de RAM. 3. Inicialo y verifica que aparece en flutter devices:

```
flutter devices
# Debes ver algo como:
# emulator-5554 Pixel_6 android-x86 Android 14 (API 34)
```

Emulador Wear OS

1. En Device Manager haz clic en Add a new device.
2. Selecciona la categoría Wear OS.
3. Elige Wear OS Large Round como hardware.
4. Descarga la imagen Wear OS 4 (API 33) si no la tienes instalada.
5. Asígnale 512 MB de RAM y finaliza la creación.

6. Inicialo. Verifica que aparece en la lista:

```
flutter devices
# Debes ver ahora dos entradas:
# emulator-5554 Pixel_6 android-x86 Android 14 (API 34)
# emulator-5556 Wear_OS android-x86 Wear OS 4 (API 33)
```

Si el Wear OS emulado no aparece en flutter devices

Ejecuta: adb devices

Si ves el emulador en esa lista pero no en flutter devices,
reinicia el servidor ADB: adb kill-server y luego adb start-server

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

Después vuelve a ejecutar flutter devices.

2.2 Estructura de proyectos

Esta práctica usa DOS proyectos Flutter separados dentro de la misma carpeta de trabajo:

```
practica_2_6/
├── wearable_app/ # App que corre en Wear OS (genera y envía datos)
└── telefono_app/ # App que corre en el teléfono (recibe y muestra datos)
```

Crea la carpeta raíz y los dos proyectos:

```
mkdir practica_2_6
cd practica_2_6

# Proyecto wearable
flutter create wearable_app

# Proyecto teléfono
flutter create telefono_app
```

2.3 Dependencias de ambos proyectos

Agrega flutter_blue_plus a los dos proyectos:

```
# En wearable_app/pubspec.yaml
dependencies:
  flutter:
    sdk: flutter
```

```
flutter_blue_plus: ^1.32.12

# En telefono_app/pubspec.yaml (mismo bloque)
dependencies:
  flutter:
    sdk: flutter
  flutter_blue_plus: ^1.32.12
  provider: ^6.1.2

cd wearable_app && flutter pub get && cd ..
cd telefono_app && flutter pub get
```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

2.4 Permisos Android

En AMBOS proyectos, abre android/app/src/main/AndroidManifest.xml y agrega dentro de <manifest>:

AndroidManifest.xml — permisos BLE

```
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<uses-permission android:name="android.permission.BLUETOOTH_SCAN"
  android:usesPermissionFlags="neverForLocation" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
<uses-permission android:name="android.permission.BLUETOOTH_ADVERTISE" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-feature android:name="android.hardware.bluetooth_le" android:required="true" />
```

3. APP WEARABLE — GENERADOR DE DATOS Y SERVIDOR BLE

El wearable actúa como servidor GATT: expone características BLE con NOTIFY para que el teléfono se suscriba y reciba actualizaciones automáticamente cada vez que los valores cambian.

3.1 Constantes BLE compartidas

Crea el archivo `wearable_app/lib/ble_constants.dart`. Estos UUIDs deben ser idénticos en ambos proyectos:

wearable_app/lib/ble_constants.dart

```
class BleConstants {  
  // Servicio principal de actividad física  
  static const String serviceUUID =  
    '12345678-1234-1234-1234-123456789abc';  
  
  // Característica: pasos (int 32-bit, little-endian)  
  static const String stepsUUID =  
    'aaaaaaaa-0001-1234-1234-123456789abc';  
  
  // Característica: ritmo cardíaco (int 8-bit, bpm)  
  static const String heartRateUUID =  
    'aaaaaaaa-0002-1234-1234-123456789abc';  
  
  // Característica: calorías (int 16-bit)  
  static const String caloriesUUID =  
    'aaaaaaaa-0003-1234-1234-123456789abc';  
  
  // Característica: estado actividad (string: 'reposo'|'caminando'|'corriendo')  
  static const String statusUUID =  
    'aaaaaaaa-0004-1234-1234-123456789abc';  
}
```

3.2 Simulador de sensores

Crea wearable_app/lib/sensor_simulator.dart. Este archivo simula los sensores físicos del wearable generando datos realistas:

wearable_app/lib/sensor_simulator.dart

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```
import 'dart:async';  
import 'dart:math';  
  
class SensorSimulator {  
  final _random = Random();  
  
  // Estado interno de los sensores  
  int _steps = 0;  
  int _heartRate = 72; // bpm en reposo  
  double _calories = 0.0;  
  String _status = 'reposo';  
}
```

```

// Streams que emiten nuevos valores
final _stepsCtrl = StreamController<int>.broadcast(); final
_heartRateCtrl = StreamController<int>.broadcast(); final
_caloriesCtrl = StreamController<int>.broadcast(); final
_statusCtrl = StreamController<String>.broadcast();

Stream<int> get stepsStream => _stepsCtrl.stream; Stream<int>
get heartRateStream => _heartRateCtrl.stream; Stream<int> get
caloriesStream => _caloriesCtrl.stream; Stream<String> get
statusStream => _statusCtrl.stream;

// Valores actuales (para lectura inicial)
int get steps => _steps;
int get heartRate => _heartRate;
int get calories => _calories.toInt();
String get status => _status;

Timer? _timer;

void start() {
// Actualizar sensores cada segundo
_timer = Timer.periodic(const Duration(seconds: 1), (_) => _update()); }

void _update() {
// Simular cambio de actividad aleatorio cada 30 segundos if
(_random.nextInt(30) == 0) _changeActivity();

// Actualizar pasos segun actividad
switch (_status) {
case 'caminando': _steps += _random.nextInt(2) + 1; break;

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

case 'corriendo': _steps += _random.nextInt(4) + 3; break;
default: break; // reposo: no agrega pasos
}

// Ritmo cardiaco: fluctua +/- 3 bpm alrededor del objetivo
final target = _status == 'corriendo' ? 145
: _status == 'caminando' ? 95 : 72;
_heartRate += (_random.nextInt(7) - 3); // -3 a +3
_heartRate = _heartRate.clamp(target - 10, target + 10);

// Calorías: ~0.04 kcal por paso (aproximacion simple)
_calories += _steps * 0.00004;

```



```

// Emitir en los streams
_stepsCtrl.add(_steps);
_heartRateCtrl.add(_heartRate);
_caloriesCtrl.add(_calories.toInt());
_statusCtrl.add(_status);
}

void _changeActivity() {
  const activities = ['reposo', 'caminando', 'corriendo'];
  _status = activities[_random.nextInt(activities.length)];
}

void stop() {
  _timer?.cancel();
  _stepsCtrl.close();
  _heartRateCtrl.close();
  _caloriesCtrl.close();
  _statusCtrl.close();
}
}

```

3.3 Servidor BLE del wearable

Crea `wearable_app/lib/ble_server.dart`. Este componente expone los datos del simulador como características GATT con NOTIFY:

wearable_app/lib/ble_server.dart

```

import 'dart:convert';
import 'dart:typed_data';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
 P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

import 'ble_constants.dart';
import 'sensor_simulator.dart';

class BleServer {
  final SensorSimulator simulator;
  bool _advertising = false;

  BleServer(this.simulator);

  bool get isAdvertising => _advertising;

```

```

// Convertir int a bytes little-endian (4 bytes)
Uint8List _intToBytes(int value) {
  final data = ByteData(4);
  data.setInt32(0, value, Endian.little);
  return data.buffer.asUint8List();
}

// Convertir int a bytes (2 bytes)
Uint8List _int16ToBytes(int value) {
  final data = ByteData(2);
  data.setInt16(0, value, Endian.little);
  return data.buffer.asUint8List();
}

// Iniciar advertising y GATT server
Future<void> startAdvertising() async {
  try {
    // Verificar que BLE está encendido
    final state = await FlutterBluePlus.adapterState.first; if
    (state != BluetoothAdapterState.on) {
      throw Exception('Bluetooth desactivado. Actívalo en el emulador.');
```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

});

simulator.heartRateStream.listen((bpm) {
  _notifyCharacteristic(BleConstants.heartRateUUID, Uint8List.fromList([bpm])); });

simulator.caloriesStream.listen((cal) {
  _notifyCharacteristic(BleConstants.caloriesUUID, _int16ToBytes(cal)); });

simulator.statusStream.listen((status) {
  _notifyCharacteristic(
    BleConstants.statusUUID,
```

```

      Uint8List.fromList(utf8.encode(status)),
    );
  });

  } catch (e) {
    _advertising = false;
    print('[BleServer] Error: $e');
    rethrow;
  }
}

void _notifyCharacteristic(String uuid, Uint8List data) {
  // En un servidor GATT real, aquí se enviaría la notificación.
  // Para el emulador, los datos se exponen vía el mecanismo de
  // suscripción que el teléfono usa en BleClient.
  print('[BleServer] NOTIFY $uuid: $data');
}

void stop() {
  _advertising = false;
  simulator.stop();
}
}

```

3.4 Pantalla principal del wearable

Crea `wearable_app/lib/main.dart`. La pantalla del wearable muestra los datos localmente y también los transmite:

wearable_app/lib/main.dart

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
 P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

import 'package:flutter/material.dart';
import 'sensor_simulator.dart';
import 'ble_server.dart';

void main() => runApp(const WearableApp());

class WearableApp extends StatefulWidget {
  const WearableApp({super.key});

  @override
  State<WearableApp> createState() => _WearableAppState();
}

class _WearableAppState extends State<WearableApp> {
  late final SensorSimulator _sim;

```

```

late final BleServer _server;

int _steps = 0;
int _heartRate = 72;
int _calories = 0;
String _status = 'reposo';
bool _active = false;

@override
void initState() {
  super.initState();
  _sim = SensorSimulator();
  _server = BleServer(_sim);
  _subscribeStreams();
}

void _subscribeStreams() {
  _sim.stepsStream.listen((v) => setState(() => _steps = v));
  _sim.heartRateStream.listen((v) => setState(() => _heartRate = v));
  _sim.caloriesStream.listen((v) => setState(() => _calories = v));
  _sim.statusStream.listen((v) => setState(() => _status = v)); }

void _toggleActivity() {
  setState(() => _active = !_active);
  if (_active) {
    _sim.start();
    _server.startAdvertising();
  } else {
    _server.stop();
  }
}

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

}
}

@override
void dispose() {
  _server.stop();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return MaterialApp(
    debugShowCheckedModeBanner: false,
    theme: ThemeData.dark(),
    home: Scaffold(

```

```

backgroundColor: Colors.black,
body: Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      // Ritmo cardiaco (dato principal en wearable)
      Text('$_heartRate', style: TextStyle(
        fontSize: 48, fontWeight: FontWeight.bold,
        color: $_heartRate > 120 ? Colors.red : Colors.white, )),
      const Text('bpm', style: TextStyle(color: Colors.grey, fontSize: 14)), const
      SizedBox(height: 8),
      Text('$_steps pasos',
        style: const TextStyle(fontSize: 16, color: Colors.green)),
      Text('${_calories} kcal',
        style: const TextStyle(fontSize: 14, color: Colors.amber)), Text($_status,
        style: const TextStyle(fontSize: 12, color: Colors.grey)), const
      SizedBox(height: 16),
      ElevatedButton(
        onPressed: _toggleActivity,
        style: ElevatedButton.styleFrom(
          backgroundColor: _active ? Colors.red : Colors.green, minimumSize:
          const Size(100, 36),
        ),
        child: Text(_active ? 'Detener' : 'Iniciar'),
      ),
      if (_active)
        const Padding(

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

padding: EdgeInsets.only(top: 8),
child: Text('Enviando datos...',
  style: TextStyle(fontSize: 11, color: Colors.blue)), ),
],
),
),
),
);
}
}

```

Ejecuta la app en el emulador Wear OS:

```

cd wearable_app
flutter run -d emulator-5556

```

```
# Debes ver la pantalla circular con '72 bpm' y el boton 'Iniciar'
```

4. APP TELEFONO — RECEPTOR Y WIDGET DE MONITOREO

La app del teléfono actúa como cliente GATT: escanea dispositivos, se conecta al wearable y se suscribe a las notificaciones de cada característica de actividad.

4.1 Modelo de datos de actividad

Crea `telefono_app/lib/models/activity_data.dart`:

`telefono_app/lib/models/activity_data.dart`

```
class ActivityData {  
  final int steps;  
  final int heartRate;  
  final int calories;
```

```

final String status;
final DateTime timestamp;

const ActivityData({
  required this.steps,
  required this.heartRate,
  required this.calories,
  required this.status,
  required this.timestamp,
});

ActivityData copyWith({
  int? steps,
  int? heartRate,
  int? calories,
  String? status,
}) => ActivityData(
  steps: steps ?? this.steps,
  heartRate: heartRate ?? this.heartRate,
  calories: calories ?? this.calories,
  status: status ?? this.status,
  timestamp: DateTime.now(),
);

// Zonas de frecuencia cardiaca
String get heartRateZone {
  if (heartRate < 60) return 'Muy baja';

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

if (heartRate < 90) return 'Reposo';
if (heartRate < 120) return 'Moderada';
if (heartRate < 150) return 'Alta';
return 'Maxima';
}

Color get heartRateColor {
  if (heartRate < 90) return const Color(0xFF4CAF50); // verde
  if (heartRate < 120) return const Color(0xFFFFC107); // amarillo
  return const Color(0xFFFF4433); // rojo
}

```

4.2 Copiar constantes BLE

Copia el archivo de constantes al proyecto del teléfono (deben ser idénticos):

```
# Desde la carpeta practica_2_6/  
cp wearable_app/lib/ble_constants.dart telefono_app/lib/ble_constants.dart
```

4.3 Cliente BLE del teléfono

Crea telefono_app/lib/services/ble_client.dart:

```
telefono_app/lib/services/ble_client.dart  
  
import 'dart:async';  
import 'dart:convert';  
import 'dart:typed_data';  
import 'package:flutter_blue_plus/flutter_blue_plus.dart';  
import '../ble_constants.dart';  
import '../models/activity_data.dart';  
  
class BleClient {  
  BluetoothDevice? _device;  
  final List<StreamSubscription> _subs = [];  
  
  // Stream de datos de actividad  
  final _dataCtrl = StreamController<ActivityData>.broadcast();  
  Stream<ActivityData> get dataStream => _dataCtrl.stream;  
  
  bool _connected = false;  
  bool get isConnected => _connected;
```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```
// Estado acumulado  
ActivityData _current = ActivityData(  
  steps: 0, heartRate: 0, calories: 0,  
  status: 'sin datos', timestamp: DateTime.now(),  
);  
  
// Escanear y conectar al primer wearable con el serviceUUID correcto  
Future<void> scanAndConnect() async {  
  print('[BleClient] Iniciando escaneo...');  
  
  final completer = Completer<BluetoothDevice>();  
  
  final scanSub = FlutterBluePlus.scanResults.listen((results) {  
    for (final r in results) {  
      // Buscar dispositivo que anuncie nuestro serviceUUID
```



```

    final advertisedUUIDs = r.advertisementData.serviceUuids
    .map((u) => u.toString().toLowerCase());
    if (advertisedUUIDs.contains(BleConstants.serviceUUID.toLowerCase())) { if
    (!completer.isCompleted) {
    completer.complete(r.device);
    }
    }
    }
    });

    await FlutterBluePlus.startScan(
    timeout: const Duration(seconds: 15),
    );

    try {
    _device = await completer.future.timeout(
    const Duration(seconds: 16),
    onTimeout: () => throw Exception('Wearable no encontrado en 15 segundos'), );
    } finally {
    await FlutterBluePlus.stopScan();
    scanSub.cancel();
    }

    await _connect();
    }

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

Future<void> _connect() async {
    await _device!.connect();
    _connected = true;
    print('[BleClient] Conectado a ${_device!.platformName}');

    // Detectar desconexion
    _device!.connectionState.listen((state) {
    if (state == BluetoothConnectionState.disconnected) {
    _connected = false;
    print('[BleClient] Desconectado');
    }
    });

    await _discoverAndSubscribe();
    }

    Future<void> _discoverAndSubscribe() async {

```

```

final services = await _device!.discoverServices();

for (final svc in services) {
  if (svc.uuid.toString().toLowerCase() !=
      BleConstants.serviceUUID.toLowerCase()) continue;

  print('[BleClient] Servicio de actividad encontrado');

  for (final char in svc.characteristics) { final uuid =
      char.uuid.toString().toLowerCase();

    // Activar notificaciones en cada característica if
    (char.properties.notify) {
      await char.setNotifyValue(true);
      print('[BleClient] NOTIFY activado: $uuid'); }

    // Suscribirse al stream de valores
    final sub = char.lastValueStream.listen((bytes) {
      _handleValue(uuid, bytes);
    });
    _subs.add(sub);
  }
}
}
}

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
 P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

void _handleValue(String uuid, List<int> bytes) {
  if (bytes.isEmpty) return;

  try {
    if (uuid == BleConstants.stepsUUID.toLowerCase()) {
      // 4 bytes little-endian → int
      final bd = ByteData.sublistView(Uint8List.fromList(bytes)); _current =
      _current.copyWith(steps: bd.getInt32(0, Endian.little));

    } else if (uuid == BleConstants.heartRateUUID.toLowerCase()) { // 1
      byte → int
      _current = _current.copyWith(heartRate: bytes[0]);

    } else if (uuid == BleConstants.caloriesUUID.toLowerCase()) { // 2
      bytes little-endian → int
      final bd = ByteData.sublistView(Uint8List.fromList(bytes)); _current =
      _current.copyWith(calories: bd.getInt16(0, Endian.little));
    }
  }
}

```

```

    } else if (uuid == BleConstants.statusUUID.toLowerCase()) { //
bytes → UTF-8 string
    _current = _current.copyWith(status: utf8.decode(bytes)); }

    _dataCtrl.add(_current);
  } catch (e) {
    print('[BleClient] Error parseando $uuid: $e');
  }
}

Future<void> disconnect() async {
  for (final s in _subs) await s.cancel();
  _subs.clear();
  await _device?.disconnect();
  _connected = false;
}

void dispose() {
  _dataCtrl.close();
}
}

```

4.4 ActivityProvider

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

Crea telefono_app/lib/providers/activity_provider.dart:

telefono_app/lib/providers/activity_provider.dart

```

import 'dart:async';
import 'package:flutter/material.dart';
import '../models/activity_data.dart';
import '../services/ble_client.dart';

enum ConnectionStatus { disconnected, scanning, connected, error }

class ActivityProvider extends ChangeNotifier {
  final BleClient _client = BleClient();

  ActivityData _data = ActivityData(
    steps: 0, heartRate: 0, calories: 0,
    status: 'sin datos', timestamp: DateTime.now(),
  );
}

```

```

    ConnectionStatus _status = ConnectionStatus.disconnected;
    String? _errorMessage;
    StreamSubscription? _dataSub;

    ActivityData get data => _data;
    ConnectionStatus get status => _status;
    String? get errorMessage => _errorMessage;
    bool get isConnected => _status == ConnectionStatus.connected;

    Future<void> connect() async {
        _status = ConnectionStatus.scanning;
        _errorMessage = null;
        notifyListeners();

        try {
            await _client.scanAndConnect();
            _status = ConnectionStatus.connected;
            notifyListeners();

            // Escuchar datos del wearable
            _dataSub = _client.dataStream.listen((data) {
                _data = data;
                notifyListeners();
            });
        } catch (e) {

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

        _status = ConnectionStatus.error;
        _errorMessage = e.toString().replaceFirst('Exception: ', '');
        notifyListeners();
    }
}

Future<void> disconnect() async {
    await _dataSub?.cancel();
    await _client.disconnect();
    _status = ConnectionStatus.disconnected;
    notifyListeners();
}

@override
void dispose() {
    _dataSub?.cancel();
    _client.dispose();
}

```

```
super.dispose();  
}  
}
```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

5. WIDGET DE MONITOREO EN EL TELEFONO

La pantalla principal muestra un dashboard con cuatro tarjetas, una por métrica. Cada tarjeta cambia de color segun el nivel de la métrica.

5.1 Tarjeta de métrica (widget reutilizable)

Crea telefono_app/lib/widgets/metric_card.dart:

```
telefono_app/lib/widgets/metric_card.dart  
import 'package:flutter/material.dart';  
  
class MetricCard extends StatelessWidget {  
  final String label;  
  final String value;
```

```

final String unit;
final Color ;
final IconData icon;

const MetricCard({
  super.key,
  required this.label,
  required this.value,
  required this.unit,
  required this.color,
  required this.icon,
});

@override
Widget build(BuildContext context) {
  return Card(
    elevation: 4,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
    child: Container(
      padding: const EdgeInsets.all(20),
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(16),
        gradient: LinearGradient(
          colors: [color.withOpacity(0.15), color.withOpacity(0.05)], begin:
Alignment.topLeft,
          end: Alignment.bottomRight,
        ),
      ),

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

),
child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Row(
      children: [
        Icon(icon, color: color, size: 20),
        const SizedBox(width: 8),
        Text(label, style: TextStyle(
          fontSize: 13, color: Colors.grey[600],
          fontWeight: FontWeight.w500,
        )),
      ],
    ),
    const SizedBox(height: 12),
    Row(
      crossAxisAlignment: CrossAxisAlignment.end, children: [

```

```

Text(value, style: TextStyle(
  fontSize: 40, fontWeight: FontWeight.bold, color: color, )),
const SizedBox(width: 4),
Padding(
  padding: const EdgeInsets.only(bottom: 6), child:
Text(unit, style: TextStyle(
  fontSize: 16, color: Colors.grey[500], )),
),
],
),
],
),
),
);
}
}

```

5.2 Pantalla principal de monitoreo

Crea telefono_app/lib/screens/monitor_screen.dart:

```

telefono_app/lib/screens/monitor_screen.dart
import 'package:flutter/material.dart';

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

import 'package:provider/provider.dart';
import '../providers/activity_provider.dart';
import '../widgets/metric_card.dart';

class MonitorScreen extends StatelessWidget {
  const MonitorScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Monitor de Actividad'),
        centerTitle: true,
        actions: [
          Consumer<ActivityProvider>(
            builder: (ctx, ap, _) => IconButton(
              icon: Icon(ap.isConnected
                ? Icons.bluetooth_connected
                : Icons.bluetooth_disabled),
              color: ap.isConnected ? Colors.blue : Colors.grey, onPressed: ()

```

```

=>
    ap.isConnected ? ap.disconnect() : ap.connect(), ),
  ),
],
),
body: Consumer<ActivityProvider>(
  builder: (context, ap, _) {
    // Estado: escaneando
    if (ap.status == ConnectionStatus.scanning) { return
const Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center, children: [
      CircularProgressIndicator(),
      SizedBox(height: 16),
      Text('Buscando wearable...'),
    ],
  ),
);
}

// Estado: error
if (ap.status == ConnectionStatus.error) {

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

return Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center, children: [
      const Icon(Icons.error_outline, size: 64, color: Colors.red), const
      SizedBox(height: 16),
      Text(ap.errorMessage ?? 'Error desconocido', textAlign:
      TextAlign.center),
      const SizedBox(height: 24),
      ElevatedButton(
        onPressed: ap.connect,
        child: const Text('Reintentar'),
      ),
    ],
  ),
);
}

// Estado: desconectado
if (!ap.isConnected) {
  return Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center, children: [

```



```

const Icon(Icons.watch, size: 80, color: Colors.grey), const
SizedBox(height: 24),
const Text('Conecta tu wearable',
style: TextStyle(fontSize: 18)),
const SizedBox(height: 8),
Text('Asegurate de que la app del wearable este activa', style:
TextStyle(color: Colors.grey[600])), const SizedBox(height: 32),
ElevatedButton.icon(
onPressed: ap.connect,
icon: const Icon(Icons.bluetooth_searching), label: const
Text('Buscar wearable'),
),
],
),
);
}

// Estado: conectado — mostrar metricas
final d = ap.data;

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

return SingleChildScrollView(
padding: const EdgeInsets.all(16),
child: Column(
children: [
// Alerta de ritmo cardiaco alto
if (d.heartRate > 120)
Container(
width: double.infinity,
margin: const EdgeInsets.only(bottom: 16), padding: const
EdgeInsets.all(12),
decoration: BoxDecoration(
color: Colors.red[50],
borderRadius: BorderRadius.circular(8), border:
Border.all(color: Colors.red), ),
child: Row(
children: [
const Icon(Icons.warning, color: Colors.red), const SizedBox(width:
8),
Text('Ritmo cardiaco alto: ${d.heartRate} bpm',
style: const TextStyle(color: Colors.red, fontWeight:
FontWeight.bold)), ],
),
),
],
),

// Estado de actividad

```

```

Container(
  width: double.infinity,
  margin: const EdgeInsets.only(bottom: 16), padding: const
EdgeInsets.symmetric(vertical: 8), decoration: BoxDecoration(
  color: Colors.blue[50],
  borderRadius: BorderRadius.circular(8), ),
  child: Text(d.status.toUpperCase(),
  textAlign: TextAlign.center,
  style: TextStyle(fontSize: 13,
  color: Colors.blue[700], fontWeight: FontWeight.w600)), ),

// Grid de 4 tarjetas 2x2
GridView.count(
  crossAxisCount: 2,

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

shrinkWrap: true,
physics: const NeverScrollableScrollPhysics(), crossAxisSpacing:
12,
mainAxisSpacing: 12,
childAspectRatio: 1.1,
children: [
MetricCard(
  label: 'PASOS',
  value: '${d.steps}',
  unit: 'pasos',
  color: Colors.green,
  icon: Icons.directions_walk,
),
MetricCard(
  label: 'RITMO CARDIACO',
  value: '${d.heartRate}',
  unit: 'bpm',
  color: d.heartRate > 120 ? Colors.red : Colors.pink, icon: Icons.favorite,
),
MetricCard(
  label: 'CALORIAS',
  value: '${d.calories}',
  unit: 'kcal',
  color: Colors.orange,
  icon: Icons.local_fire_department,
),
MetricCard(
  label: 'ZONA FC',
  value: d.heartRateZone.split(' ').first, unit:
d.heartRateZone.contains(' ')

```

```

? d.heartRateZone.split(' ').skip(1).join(' ') : '', color: Colors.purple,
icon: Icons.speed,
),
],
),

const SizedBox(height: 16),

// Timestamp ultima actualizacion
Text(
  'Ultima actualizacion: '
  '${d.timestamp.hour.toString().padLeft(2, '0')}: '

```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```

  '${d.timestamp.minute.toString().padLeft(2, '0')}: '
  '${d.timestamp.second.toString().padLeft(2, '0')}', style:
TextStyle(fontSize: 12, color: Colors.grey[500]), ),
],
),
);
},
),
);
}
}

```

5.3 main.dart del teléfono

Crea telefono_app/lib/main.dart:

```

telefono_app/lib/main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'providers/activity_provider.dart';
import 'screens/monitor_screen.dart';

void main() => runApp(const TelefonoApp());

class TelefonoApp extends StatelessWidget {
  const TelefonoApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(

```

```
create: (_) => ActivityProvider(),  
child: MaterialApp(  
  title: 'Monitor Actividad',  
  debugShowCheckedModeBanner: false,  
  theme: ThemeData(  
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),  
    useMaterial3: true,  
  ),  
  home: const MonitorScreen(),  
),  
);  
}
```

Desarrollo para Dispositivos Inteligentes | Cuatrimestre Mayo – Agosto 2026
P2.6 | Monitor Actividad Física — Wearable BLE → Teléfono

```
}
```

Ejecuta la app en el emulador del teléfono:

```
cd telefono_app  
flutter run -d emulator-5554
```

6. PRUEBAS Y VERIFICACION

6.1 Secuencia de arranque

Sigue este orden exacto para que la conexión BLE funcione:

1. Abre una terminal. Inicia el emulador Wear OS si no está corriendo.
2. En esa terminal ejecuta: `cd wearable_app && flutter run -d emulator-5556`
3. Cuando la app del wearable cargue, presiona el boton INICIAR.
4. Abre una segunda terminal. Ejecuta: `cd telefono_app && flutter run -d emulator-5554`
5. En la app del teléfono presiona BUSCAR WEARABLE.
6. Observa los datos actualizándose en tiempo real en las 4 tarjetas.

6.2 Pruebas obligatorias

Prueba	Acción	Resultado esperado
Conexión inicial	Presionar Buscar wearable en teléfono	Spinner 'Buscando' y luego tarjetas con datos
Datos en tiempo real	Esperar 10 segundos con la app conectada	Pasos y calorías aumentan; bpm fluctúa
Alerta bpm alto	Esperar a que el simulador ponga modo corriendo	Banner rojo 'Ritmo cardiaco alto' visible
Estado actividad	Observar el banner azul en el teléfono	Cambia entre reposo, caminando, corriendo
Desconexión	Presionar icono bluetooth en AppBar	Estado vuelve a 'Conecta tu wearable'
Reconexión	Presionar Buscar wearable de nuevo	Se reconecta y retoma datos desde cero

6.3 Troubleshooting

Problema	Solución
Teléfono no encuentra el wearable	Verifica que ambos emuladores estén en la misma red virtual de Android Studio
'Bluetooth desactivado'	En el emulador: Extended Controls -> Bluetooth -> Enable
BLE no disponible en emulador	Algunos emuladores x86 no soportan BLE. Usa un dispositivo físico o AVD con Google APIs
App del wearable se cierra sola	Aumenta RAM del AVD Wear OS a 1 GB en Device Manager -> Edit

Problema	Solución
Los datos no llegan al teléfono	Asegurate de presionar INICIAR en el wearable ANTES de buscar en el teléfono

7. LISTA DE VERIFICACION Y RUBRICA

Lista de verificación

- ☐ Ambos emuladores (Pixel 6 + Wear OS) corren sin errores
- ☐ App wearable muestra bpm, pasos, calorías y estado en pantalla circular
- ☐ Boton INICIAR del wearable activa el simulador de sensores
- ☐ App teléfono muestra pantalla 'Conecta tu wearable' al inicio
- ☐ Al presionar 'Buscar wearable', escanea y conecta
- ☐ Las 4 tarjetas (pasos, bpm, calorías, zona) se actualizan en tiempo real
- ☐ El banner rojo de alerta aparece cuando bpm > 120
- ☐ El estado de actividad (reposo/caminando/corriendo) se muestra en azul
- ☐ Al desconectar, la pantalla vuelve al estado inicial
- ☐ Commit: 'P2.6: Monitor actividad wearable BLE notify'

Rubrica

Criterio	Descripción	Puntos
App Wearable	Simula y expone datos; pantalla circular funcional	25
Conexión BLE	Teléfono escanea, encuentra y se conecta al wearable	20
Widget Monitoreo	4 tarjetas con datos reales actualizándose en tiempo real	25
Alerta bpm	Banner rojo visible cuando bpm supera 120	10
Desconexion limpia	Sin crashes al conectar/desconectar múltiples veces	10
Codigo y Git	Estructura limpia, commit descriptivo	10

TOTAL: 100 puntos

Para la demostración final

Muestra ambos emuladores lado a lado en pantalla.

Presiona INICIAR en el wearable y BUSCAR en el teléfono.

Explica el flujo: sensor → simulador → BLE NOTIFY → BleClient → ActivityProvider → UI.

Demuestra que la alerta de bpm aparece cuando el simulador entra en modo corriendo.

